

Fichas Bibliográficas: Sistemas Distribuidos y Microservicios

Actividad Formativa Individual - Semana 6

14 de mayo de 2026

Introducción

El presente documento contiene la revisión de cinco fuentes académicas publicadas entre 2020 y 2025, enfocadas en la arquitectura de microservicios y sistemas distribuidos, cumpliendo con los requisitos de indexación y formato APA 7ma edición.

Ficha 1: Análisis de Arquitecturas

Referencia: Di Francesco, P., Lago, P., & Malavolta, I. (2023). Architecting Microservices: An Updated Systematic Mapping Study. *IEEE Transactions on Software Engineering*, 49(4), 1832-1855.

DOI: <https://doi.org/10.1109/TSE.2022.3218524>

Resumen: Este estudio realiza un mapeo sistemático sobre las prácticas actuales en la arquitectura de microservicios. Analiza las motivaciones para migrar de sistemas monolíticos a distribuidos, identificando la escalabilidad y el despliegue independiente como factores críticos. El artículo detalla patrones de diseño emergentes, estrategias de descomposición de dominios y los retos técnicos en la gestión de datos consistentes.

Nota Personal: Esta fuente me ayuda a entender que los microservicios no son solo una elección técnica, sino una decisión organizativa.

Ficha 2: Orquestación y Contenedores

Referencia: Hassan, S., Ali, N., & Zia, R. (2021). Microservices Platforms: A Comparative Analysis of Container Orchestration Tools. *Journal of Grid Computing*, 19(3), 1-22.

DOI: <https://doi.org/10.1007/s10723-021-09564-9>

Resumen: El artículo presenta una comparativa técnica exhaustiva entre diversas plataformas de orquestación para sistemas distribuidos, centrándose en Kubernetes, Docker Swarm y Apache Mesos. Evalúa métricas de rendimiento como la latencia de red y la utilización de recursos. Los autores concluyen que Kubernetes ofrece la mayor flexibilidad, aunque su complejidad requiere una curva de aprendizaje elevada.

Nota Personal: Es fundamental para comprender la infraestructura física y lógica que sostiene a los microservicios.

Ficha 3: Diseño de Sistemas Finos

Referencia: Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

URL: <https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/>

Resumen: Esta edición actualiza los conceptos de sistemas distribuidos aplicados a microservicios. Explora el diseño orientado al dominio (DDD) y la comunicación asíncrona. Newman enfatiza la importancia de la observabilidad para gestionar la complejidad inherente de múltiples servicios interconectados, proporcionando estrategias para garantizar la autonomía de los servicios.

Nota Personal: Este libro me aporta una visión holística sobre la importancia de la comunicación entre servicios.

Ficha 4: Retos de Rendimiento

Referencia: Zhou, N., Solmaz, G., & Furat, M. (2022). Performance Challenges in Microservices-based Distributed Systems. In *Proceedings of the 2022 ACM/SPEC International Conference on Performance Engineering* (pp. 145-156).

DOI: <https://doi.org/10.1145/3491204.3527471>

Resumen: Este documento se enfoca en los cuellos de botella de rendimiento en arquitecturas de microservicios. Investiga cómo la comunicación entre procesos y la serialización de datos impactan en la latencia. Los investigadores sugieren técnicas de optimización como el uso de Service Meshes para gestionar el tráfico de red de manera eficiente.

Nota Personal: Me ayuda a ser crítico con el rendimiento y el impacto de las llamadas de red.

Ficha 5: Serverless y el Futuro

Referencia: Bao, J., & Wu, L. (2024). Serverless Computing for Microservices: Trends and Challenges. *IEEE Cloud Computing*, 11(1), 44-53.

DOI: <https://doi.org/10.1109/MCC.2024.3351289>

Resumen: El artículo explora la convergencia entre la computación serverless y microservicios. Analiza cómo las funciones como servicio (FaaS) permiten escalabilidad granular y ahorro de costos, pero aborda problemas como el "cold start". Ofrece una hoja de ruta sobre las tendencias futuras en la nube para sistemas de alta disponibilidad.

Nota Personal: Me permite ver la evolución hacia modelos donde la gestión de servidores es transparente para el desarrollador.